

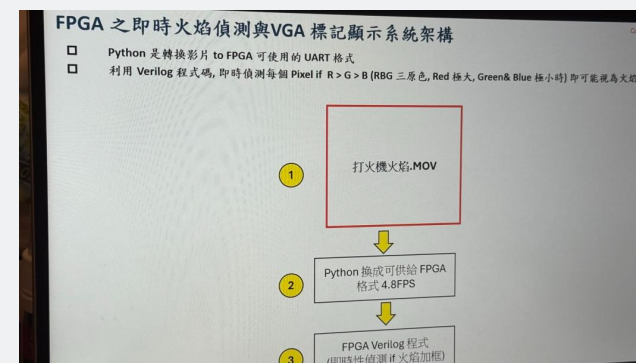
FPGA 即時火焰偵測 與 VGA 標記顯示系統

Python 影片轉換 · UART 傳輸 · Verilog 即時判斷 · VGA 紅框顯示

160 × 120 影像解析度 / 約 4.8 FPS 傳輸目標

專題簡報

Basys3 FPGA



系統目標與展示重點

將影片資料轉成 FPGA 可接收格式，並在 VGA 畫面中即時標記火焰區域。

160×120

輸入影像解析度

4.8 FPS

UART 傳輸後顯示速度

紅框

火焰區域即時標記

核心概念

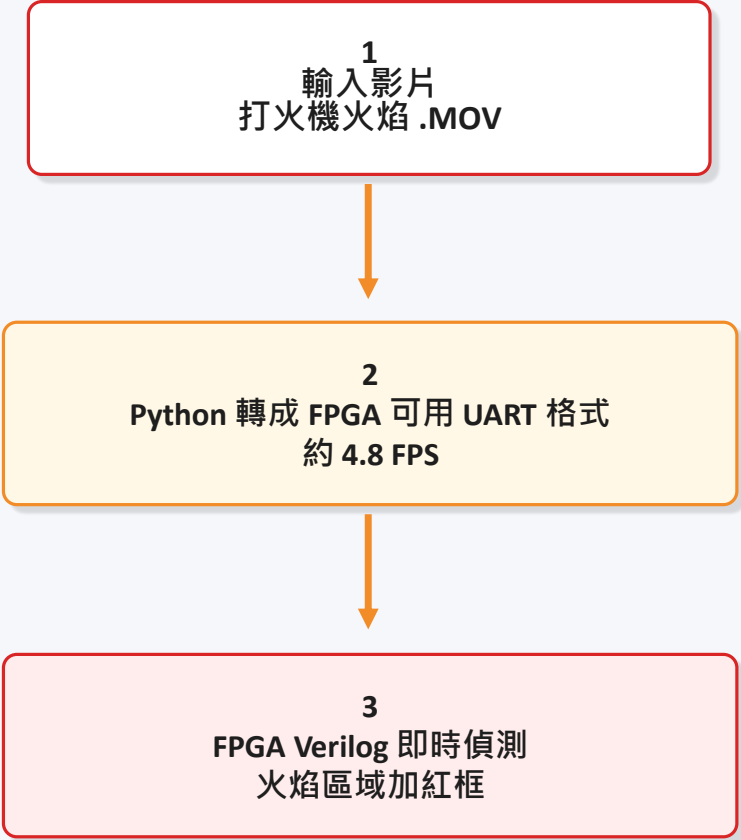
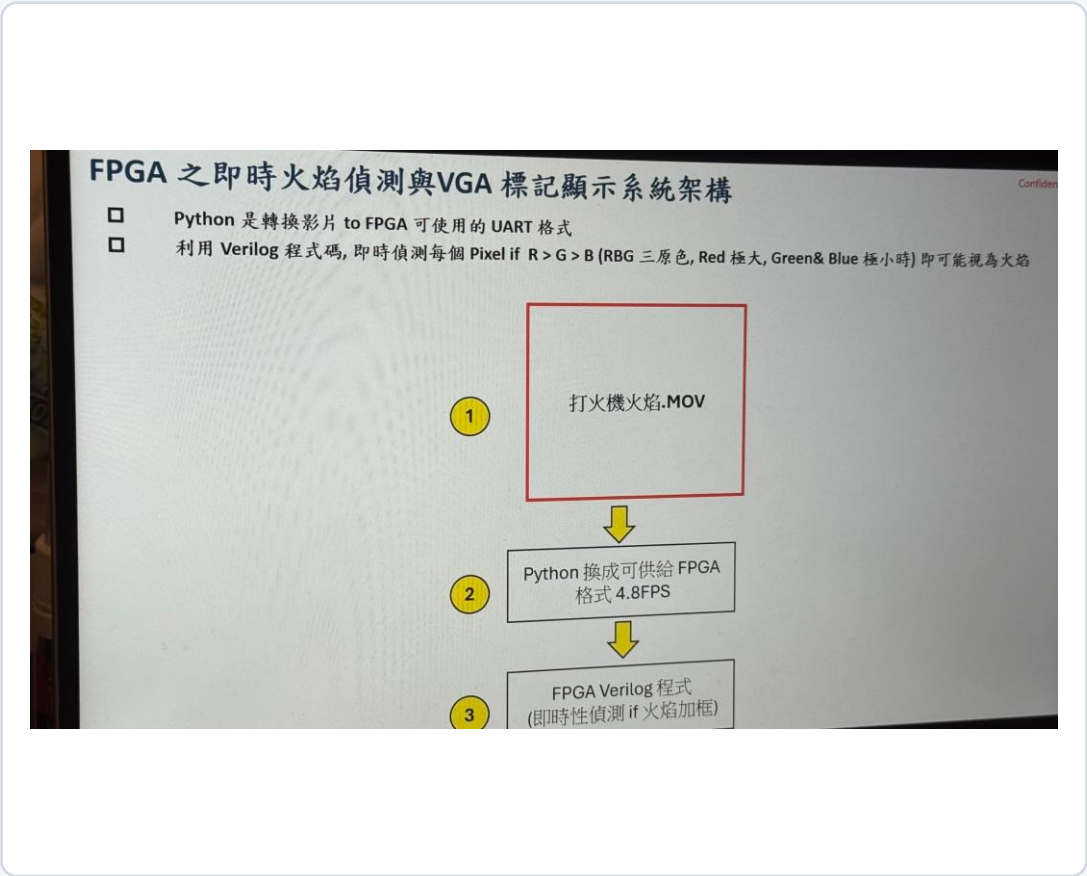
$R > G > B$ 時
判斷可能為火焰像素

- Python 負責影片降解析度、格式轉換與 UART 傳送。
- FPGA 端使用 Verilog 逐像素判斷火焰，產生標記座標。
- VGA 模組顯示原始影像，同時在火焰區域外圍畫紅框。

設計方向：讓運算留在 FPGA，降低電腦端負擔，並維持即時顯示。

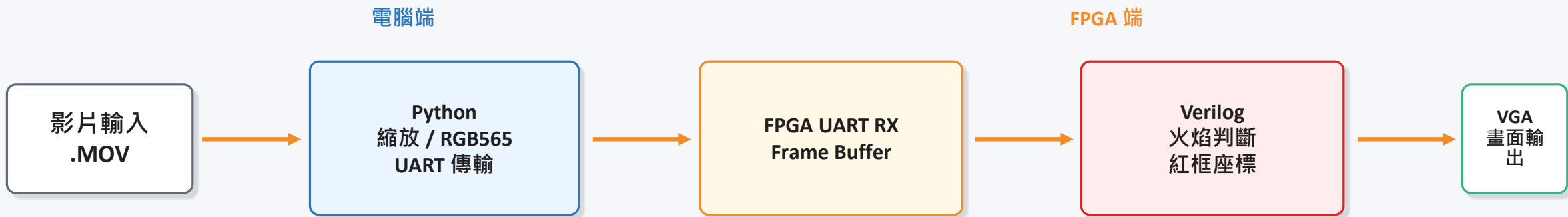
原始架構草圖整理

依據你提供的照片內容，整理成簡報可呈現的版本。



整體系統架構

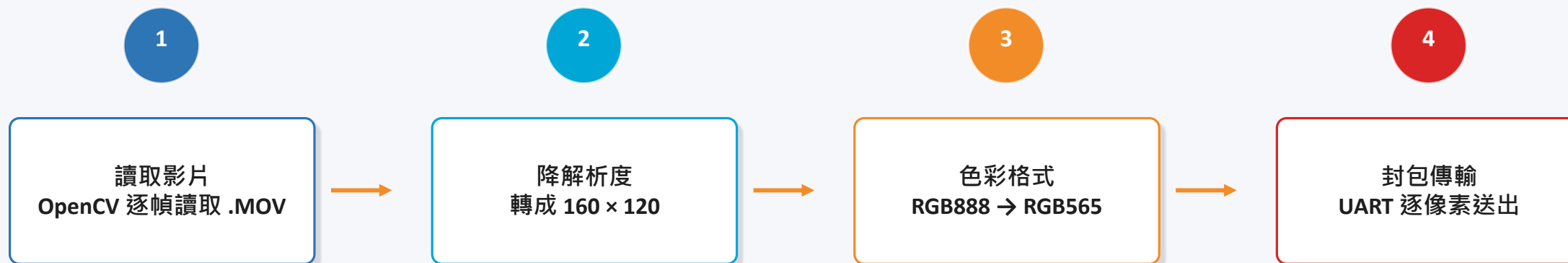
資料從影片進入 Python，再經 UART 傳到 FPGA，最後由 VGA 顯示紅框結果。



Pixel Stream：每個像素依序處理，不需要整張圖一次運算完成

Python 影片轉換流程

目標是讓影片變成 FPGA 可以穩定接收的像素串流。



建議 Python 處理重點

固定解析度、固定像素格式、固定幀同步訊號，讓 FPGA 端比較容易判斷每一張 frame。

UART 封包概念

Header + Frame ID + RGB565 Pixel Data · FPGA 收到完整 frame 後更新 VGA buffer。

FPGA Verilog 即時火焰判斷

使用 RGB 三原色關係進行快速偵測，硬體端可逐像素即時運算。

```
always @(posedge clk) begin
    if (pixel_valid) begin
        flame_pixel <= (R > G) && (G > B)
                        && (R > TH_R);

        if (flame_pixel) begin
            min_x <= min(min_x, x);
            max_x <= max(max_x, x);
            min_y <= min(min_y, y);
            max_y <= max(max_y, y);
        end
    end
end
```

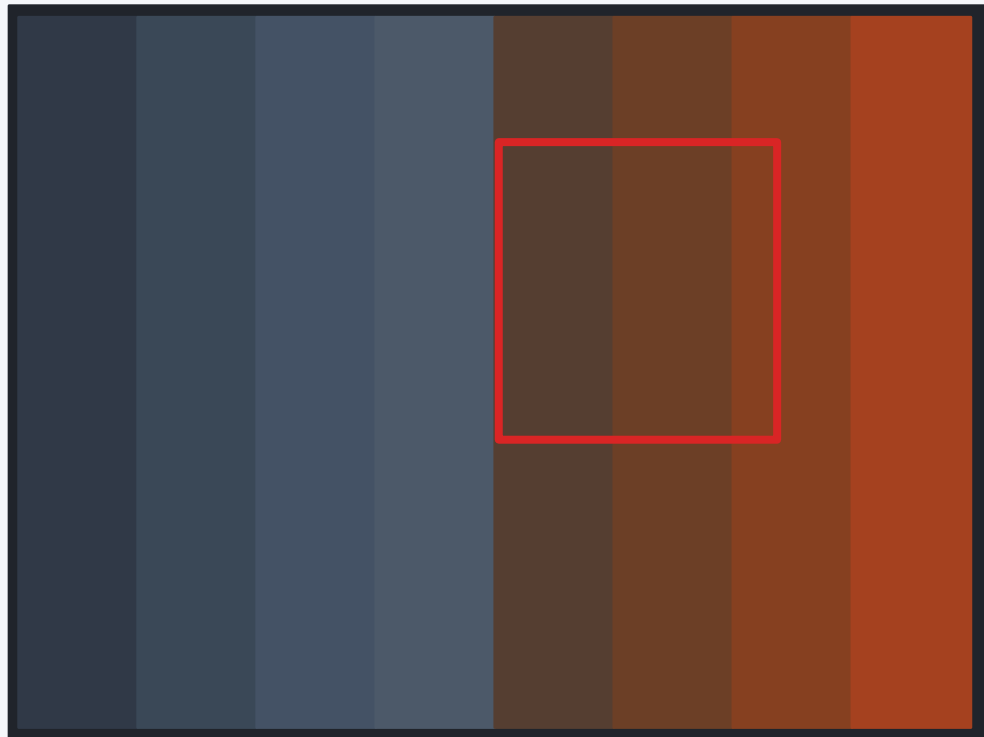
判斷條件

- R 亮度最高：火焰偏紅、偏橘。
- G 介於中間：避免把純紅物體全部誤判。
- B 最小：降低藍色背景或冷色光干擾。
- 用 min/max 座標形成紅框，不需要輸出每個 XY 座標。

輸出：frame_box_valid、min_x、max_x、min_y、max_y

VGA 顯示與紅框疊加

VGA 端只需判斷目前掃描座標是否落在框線上，即可畫出火焰標記。



VGA 畫面示意

顯示邏輯

- VGA 掃描座標產生 x / y 。
- 讀取 `buffer` 中對應像素 RGB。
- 若座標位於紅框邊界，輸出紅色。
- 否則輸出原本影像像素。

框線判斷： $x == \text{min_x}$ 或 $x == \text{max_x}$ 或 $y == \text{min_y}$ 或 $y == \text{max_y}$

傳輸速度與 FPS 估算

160×120 解析度可降低 UART 負擔，讓 FPGA 端有較穩定的畫面更新。

19,200

pixels / frame

38,400

bytes / frame (RGB565)

≈1.84 Mbps

4.8 FPS 所需序列資料量

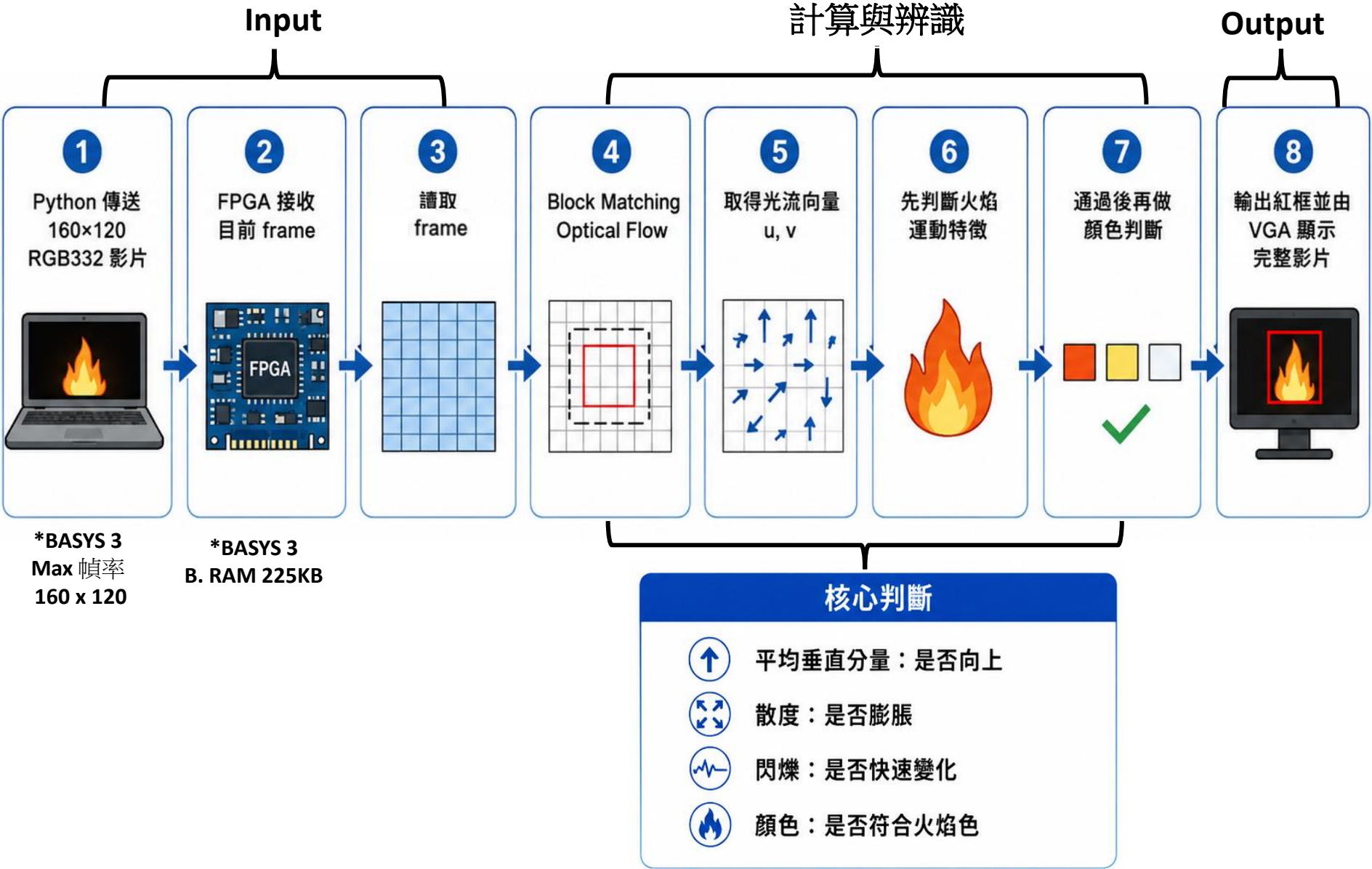
計算式： $160 \times 120 \times 2 \text{ bytes} \times 4.8 \text{ FPS} \times 10 \text{ UART bits} \approx 1.84 \text{ Mbps}$

改善方向

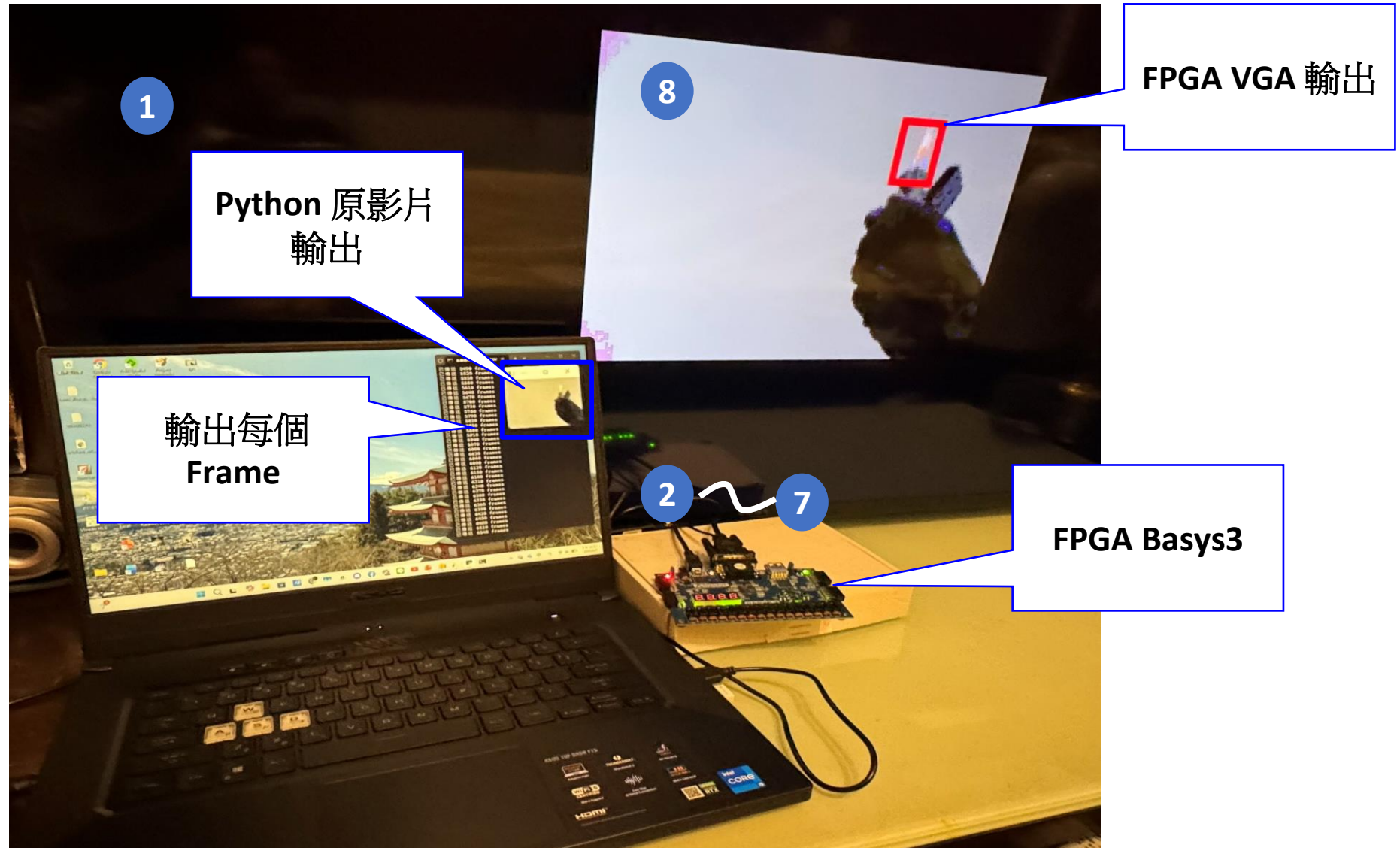
● 提高 UART baud rate 或改用壓縮傳輸。

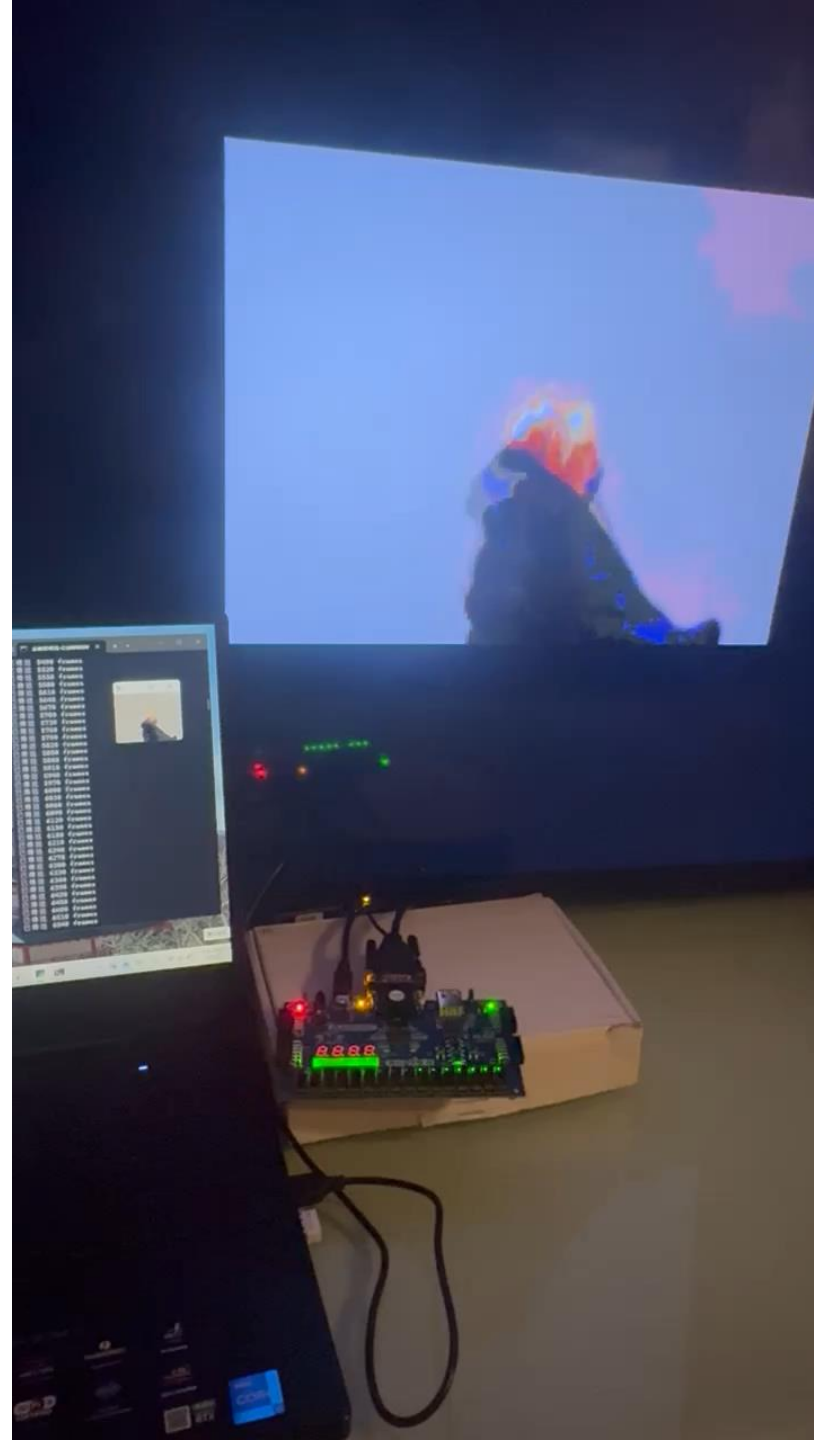
● 只傳火焰紅框座標，可大幅降低資料量。

FPGA 火焰偵測系統 流程圖



FPGA 火焰偵測系統 硬體/ 軟體架構





結論

此架構可完成影片到 FPGA 的即時顯示流程，並透過簡單 RGB 條件在硬體端快速找出火焰區域。

即時性

像素串流進入 FPGA 後即可同步處理。

低成本

使用 RGB 條件與座標範圍，不需複雜模型。

可擴充

未來可加入面積門檻、連通區塊或光流判斷。

Q & A